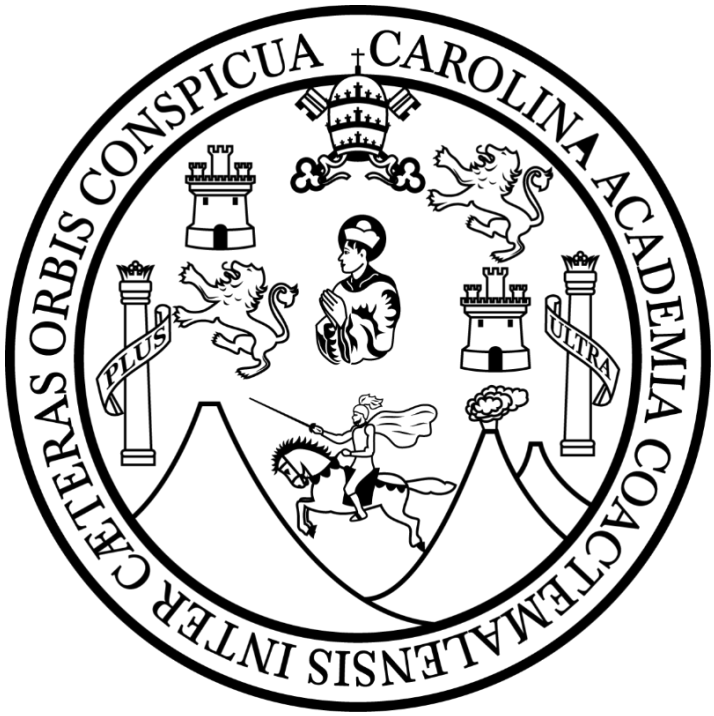
Universidad de San Carlos de Guatemala

Facultad de Ingeniería
Curso: Laboratorio
Estructuras de Datos
Sección: A
Ing. Edgar Ornelis

Tutor Académico: Giovanni Saul Concohá Cax



Proyecto Fase 1

CinemaUsac

Nombre: Brandon Denilson Polanco Guzman

Carné: 202308386

Fecha:19/08/2026

Introducción:

El documento explica la estructura y el funcionamiento de CinemaUSAC, un sistema para gestionar la administración de un cine algunos puntos a detallar de lo que contine una cartelera para reservar asientos, desarrollado en C++ con una interfaz gráfica hecha con Qt. El propósito principal de este software es hacer más eficiente el trabajo en una sala de cine y mejorar la experiencia de las personas que lo usan.

Desde el punto de vista técnico, el proyecto se creó desde cero sin usar herramientas de la biblioteca estándar de C++ para manejar colecciones manejando el uso de estructuras dinámicas implementadas a mano. El sistema hace uso de algunas estructuras de datos propias, como Árboles Binarios de Búsqueda, Matrices Dispersas Ortogonales y varios tipos de Listas Enlazadas.

Estructuras de datos Implementadas:

Árbol binario de búsqueda (BST)

Para la implementación Creé un árbol binario con nodos que representan películas. Cada nodo tiene dos punteros, uno para la izquierda y otro para la derecha. Cuando se inserta una película, se ordena según su código alfanumérico. Los códigos menores se van a la izquierda y los mayores se van a la derecha.
Decisión de diseño:

Se hizo uso de esta estructura porque hace que la búsqueda sea mucho más rápida cuando alguien busca una película por su código. Además, cuando se recorre el árbol de manera ordenada, se obtiene automáticamente el catálogo de películas en orden, lo que facilita mostrar los datos en la interfaz gráfica sin necesidad de ordenarlos de nuevo.

Su Manejo en código Utiliza funciones recursivas para controlar la navegación e inserción de películas. Cuando se inserta una película, el código compara el código de la película con los existentes y se mueve por el árbol hasta encontrar un lugar vacío donde agregar la nueva película.

En la gestión de memoria Cada película se crea de manera dinámica. Cuando se cierra el programa, se elimina la memoria utilizada por las películas de manera segura. Se hace un recorrido especial por el árbol para asegurarse de que se eliminen todas las películas en el orden correcto, empezando por las más bajas y terminando por la raíz del árbol. Esto asegura que no queden restos de datos en la memoria.

Matriz Dispersa:

Se creo una matriz especial basada en nodos que tienen cuatro punteros: arriba, abajo, izquierda y derecha. Esta matriz se gestiona con cabeceras dinámicas para las filas y las columnas. El nodo principal, que es como el centro de todo, se encuentra en la coordenada (-1, -1).

La razón por la que se eligió esta forma de hacer las cosas es que las matrices tradicionales de dos dimensiones desperdician mucho espacio de memoria. Esto se debe a que reservan espacio para todos los elementos, incluso si no se van a utilizar. Esta matriz dispersa es mucho más eficiente en cuanto a memoria se refiere. Solo crea un nodo en la memoria cuando alguien confirma la reserva de un asiento. Si la sala está vacía, la estructura solo mantiene las cabeceras básicas.

En el código, la cuadrícula se crea enlazando punteros bidireccionales. Cuando se reserva un asiento, el algoritmo busca las cabeceras de fila y columna correspondientes, o las crea si no existen. Luego, el nuevo nodo ajusta los punteros de sus vecinos horizontales y verticales.

Para gestionar la memoria, como solo hay una matriz activa por función, implementé un método para reiniciarla. Este método elimina primero los nodos internos, que son los asientos, y luego los nodos cabecera. De esta manera, se garantiza que la memoria se libere correctamente cada vez que el administrador crea una nueva función, lo que evita sobrecargar la memoria RAM

Lista Circular Simple anidada con Lista Doble:

Se diseño un modelo de lista de listas para almacenar las promociones. La estructura principal es una lista circular simple donde cada elemento apunta al siguiente. La última parte de la lista siempre apunta al inicio. Cada elemento de la lista contiene otra lista doblemente enlazada que gestiona los beneficios como descuentos y ofertas especiales.

Se hizo este diseño porque la lista circular me permite mostrar las promociones en la interfaz de usuario como un carrusel que no termina. La lista doblemente enlazada dentro de cada elemento me da la flexibilidad de moverme hacia adelante y hacia atrás entre los beneficios, y también puedo agregar o eliminar beneficios específicos sin afectar el resto del sistema.

Al programar, se manejó con cuidado la lógica de inserción en la lista circular. Siempre tengo que asegurarme de que el último elemento agregado apunte al inicio de la lista. Para mostrar los datos en la interfaz, uso ciclos que se repiten hasta que vuelvo al inicio.

Para gestionar la memoria, destruyo todo en orden. Cuando elimino la lista circular, recorro cada elemento y libero su memoria. Al eliminar un elemento, también libero la memoria de su lista de beneficios interna. Esto se hace de manera automática para asegurarme de que no quede memoria sin usar.

Lista Circular Doblemente Enlazada:

Implementación: Creé una lista en la que cada parte tiene dos flechas, una que apunta a la siguiente parte y otra que apunta a la parte anterior. La última parte se conecta con la primera en ambos sentidos.

Su diseño Esta forma de organizar las cosas es perfecta para una cola de atención administrativa. Permite a la persona que administra moverse libremente entre las solicitudes de los clientes, avanzando o retrocediendo sin tener que preocuparse por el principio o el final de la lista.

Para el Manejo en código Cuando se agrega una nueva parte, se actualizan cuatro conexiones al mismo tiempo: la flecha que apunta a la siguiente parte de la última, la flecha que apunta al principio, y las dos flechas de la nueva parte para cerrar el círculo.

Su Gestión de memoria Cuando se elimina esta estructura, el programa deberá realmente romper el círculo. Se guarda una referencia a la primera parte y se avanza una por una, eliminando las partes siguientes mediante una variable auxiliar, hasta que se regresa al punto de inicio, y finalmente se elimina la primera parte.